

WILEY

INTERNATIONAL
TRANSACTIONS
IN OPERATIONAL
RESEARCHIntl. Trans. in Op. Res. 32 (2025) 244–269
DOI: 10.1111/itor.13276

Solving a multiobjective professional timetabling problem using evolutionary algorithms at Mandarin Academy

Mounir Hafsa^{a,b,*} , Pamela Wattebled^a, Julie Jacques^b and Laetitia Jourdan^b^aMandarine Academy, Lille 59000, France^bUniversité de Lille, CNRS, Centrale Lille, UMR 9189 - CRISTAL F-59000 Lille, FranceE-mail: mounir.hafsa@mandarine.academy [Hafsa]; pamel.wattebled@mandarine.academy [Wattebled];
julie.jacques@univ-lille.fr [Jacques]; laetitia.jourdan@univ-lille.fr [Jourdan]

Received 26 January 2022; received in revised form 31 January 2023; accepted 9 February 2023

Abstract

In this paper, we propose a method for solving a real-world timetabling problem at Mandarin Academy. The primary motivation for this work is to provide an automated professional course scheduling tool to replace the time-consuming task of manually creating timetables that are constantly incorrect. Following a review of both scientific literature and company requirements, a mathematical model of the problem is provided, which includes 18 constraints (hard/soft) and five objectives, two of which are competing. We test a handful of multi-objective evolutionary algorithms (MOEA's) starting with the non-dominated sorting genetic algorithm (NSGA II and NSGA III), the multi-objective evolutionary algorithm based on decomposition (MOEA/D), the indicator-based evolutionary algorithm and finally the strength Pareto evolutionary algorithm. Two custom genetic operators (mutation and crossover) are proposed and compared to conventional operators (PMX and swap mutation). To obtain elite configurations, a tuning phase involving all of the aforementioned algorithms is carried out. Experiments were divided by problem size, with three to five objectives tested. Experiments include the use of real-world data from the company's catalog. This dataset was made available to the scientific community to serve as a testing ground for professional course scheduling, an underexploited field of scheduling. We discuss findings, including a comparison of each algorithm's performance using various metrics, as well as convergence graphs and population evolution.

Keywords: evolutionary algorithms; multi-objective optimisation; timetabling

1. Introduction

In times of rapidly changing working environments, it is crucial to keep employees' knowledge up to date. This is where the use of digital technologies in the workplace becomes critical to the

*Corresponding author.

© 2023 The Authors.

International Transactions in Operational Research published by John Wiley & Sons Ltd on behalf of International Federation of Operational Research Societies.

This is an open access article under the terms of the Creative Commons Attribution-NonCommercial-NoDerivs License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

success of a business. Digital learning provides cost-effective courses, easy-to-access materials, and a flexible learning environment for all employees. A corporate training program that combines digital learning with formal learning is often the best option, with some classes delivered online and others delivered in real classrooms in specific locations with competent trainers.

Training programs follow a specific plan or “timetable” to ensure that each event takes place in the appropriate room and at the appropriate time. This is related to academic course scheduling problems, which consist of scheduling classes in a limited number of rooms/time slots while respecting resource restrictions (Lewis, 2008). The timetabling problem has many different variants (exams, courses, university classes), and it has a wide variety of applications (transportation: Caprara et al., 2002; dos Santos and Borenstein, 2024, healthcare Burke et al., 2002; manufacturing: Zhu and Li, 2011, etc). The university timetabling problem has received much attention in all of these categories in recent decades.

Because both course scheduling and staff rostering are NP-complete problems (Christofides, 1979), many techniques for solving them have been proposed (Burke et al., 2007). Metaheuristic methods, such as genetic algorithms (GAs) (Derigs and Jenal, 2005), simulated annealing (Bellio et al., 2016), tabu search (Burke et al., 2003), and other approaches that tend to be inspired by a natural phenomenon (Eley, 2006; Hoos et al., 2022) have been used in a variety of works. Metaheuristics are problem-independent techniques that can be applied to a broad range of problems and not just scheduling.

The problem handled in this article falls under the professional course scheduling category, which has similar features to academic timetabling (Haase et al., 1998):

- *Objectives*: This work handles five competing objectives, which can be further extended with more objectives.
- *Planning window*: This work deal with long planning windows (months to years). Some similar academic examples, such as the International Timetabling Competition 2019, focus on an extended duration (semester).
- *Feasibility*: Events may remain unscheduled due to multiple circumstances (e.g., lack of available resources, holidays, etc).
- *Locations*: Another feature of this work is the consideration of resources located in other locations (national/international).

To our knowledge, no other works share similar features with the problem discussed in this work.

There are a few works that fit into this category of scheduling problems: Derigs and Jenal (2005) utilized a multiobjective genetic algorithm to reduce expenses and increase the number of planned events at the Ford Service Organization. The authors created a web-based decision support system that allows planners to generate, analyze, and compare the system’s suggested timetables. Their problem involves scheduling courses and trainers over 3 months with eight hard constraints and one soft constraint. The first step in their approach is the use of a construction heuristic that starts with an empty schedule and a list of ordered courses. The algorithm begins by determining a suitable room’s start/end time, then checks for trainer availability, and finally equipment. If everything is available, a course schedule is created. Any infeasible course remains unscheduled. The second step involves GAs, where a timetable is encoded by two chromosomes (course information and resources). The authors used random and swap mutation, along with the partially mapped crossover

(PMX), the cycle crossover (CX), and the order crossover (OX-OX2). To test their approach, some historical data were used but lacks in available resources (small number of trainers and rooms).

By combining integer programming -based models with large neighborhood search, Czibula et al. (2017) solved a professional course timetabling problem at the Australian electricity provider “Ausgrid.” Their work aims to minimize two objectives (the number of students not accommodated and the number of room swaps) and to maximize one objective (the rental revenue of classrooms). Despite the difficulty of dealing with a large problem using exact methods, the authors began by decomposing it into multiple subproblems, one for class scheduling and one for trainer rostering. Data from Ausgrid was used to evaluate their approach, but, as with previous work (Derigs and Jenal, 2005), their testing instance was small (low number of courses, rooms, and time periods).

To maximize the profit margin of course schedules at Lufthansa Technical Training Center, Haase et al. (1998) combined a construction heuristic with tabu search. Their objective function was to maximize the profit margin induced by a schedule, aside from that objective, nine other objectives were considered but not used in their prototype. Their approach starts with one of the multiple strategies available for selecting courses and tutors. Tabu search is then used to construct several solutions until a feasible one is found. For the test phase, the planning horizon was restricted to 6 months which is a longer time frame compared to previous works.

This paper investigates and solves a multiobjective real-world professional timetabling problem at Mandarin Academy. After conducting a literature review and collecting different constraints from decision-makers, an initial construction heuristic was designed and implemented to offer fast valid solutions. These solutions are improved by using a multitude of evolutionary algorithms starting with the non-dominated sorting genetic algorithm (*NSGAII* and *NSGAIII*), the multi-objective evolutionary algorithm based on decomposition (*MOEA/D*), the indicator-based evolutionary algorithm (*IBEA*), and finally the strength Pareto evolutionary algorithm (*SPEA2*). We proposed the Mandarin Academy Professional timetabling crossover operator *MAPT_{co}* and *MAPT_{mo}* for mutation as two custom genetic operators to be compared against the partially mapped crossover *PMX* and swap mutation *Swap*. These two classic operators are widely used in scheduling/permutation problems. Elite configurations of each algorithm were found using the i-race package. Experiments using Mandarin Academy’s historical data are discussed where we highlight the differences between algorithms and select the best-performing ones.

This work is an extension of a previously published work (Hafsa et al., 2021), the main differences and additional contributions are as follows:

- complete mathematical modelization of the problem,
- implementation and comparison of additional algorithms (*SPEA 2*, *IBEA*, and *MOEA/D*) to the already studying ones (*NSGA II* and *NSGA III*),
- proposal and study of additional objectives and new stopping criteria that correspond to real-world scenarios,
- realization of a detailed performance analysis of each approach across various datasets, and
- proposal of a new timetabling dataset (Hafsa, 2022) for the research community with the best solutions.

2. Problem description and mathematical model

2.1. Mandarin Academy: Scheduling problem

Mandarin Academy is an Ed-tech company that contributes to workplace digitization by making new technologies more accessible and usable for all employees. The company invests in innovation to improve the effectiveness of training in terms of skills, time, and budget. The combination of both digital platforms like massive open online courses and personalized support improves access to a better learning experience. Recently, Mandarin Academy introduced DiLeap Logistic, a shared decision support system that enables planners to fully manage timetables. Each partner using the platform has access to their course catalog, objectives, and constraints. Before DiLeap Logistic, timetable construction was done manually and took an average of 3–6 weeks (30 workers) to produce an initial timetable covering a year of training programs. DiLeap Logistic optimizes this task by employing a greedy algorithm that takes a list of events to be planned, along with their start/end dates and locations, and automatically generates a timetable. Automating the planning process makes it easier to rent a room, notify teachers and employees about upcoming sessions, and manage the company's resources. Because the planner can confirm the software proposals, courses will be scheduled only after the decision-maker approves them. Alerts about conflicts or noncompliance with constraints are also displayed to give the planner a sense of what happened during the planning process.

Despite its advantages, the currently used automatic timetabling tool in DiLeap Logistic has several limitations: (1) it does not follow the owner's defined restrictions (it can mobilize resources that are unavailable or in the incorrect location) and (2) it does not handle multiobjectives (only a single objective which is maximizing the number of planned training sessions). In the following section, we provide a mathematical description of constraints and objectives.

2.2. Mathematical modeling

In this section we give a brief description of multi-objective combinatorial optimization problems (MOCOPs), then we outline the features of each entity involved in the planning process and define what a valid solution is for our case. Finally, we enumerate constraints (hard/soft) and objectives functions.

MOCOPs can be defined as optimizing $F(x) = (f_1(x), f_2(x), \dots, f_n(x))$, where $x \in F_{sol}$, n is the number of objectives ($n \geq 2$), x is a vector of decision variables, F_{sol} is a set of feasible solutions, and $f_i(x)$ depicts objectives that we want to minimize/maximize (Coello et al., 2010). Unlike single-objective optimization, results are not a single solution but a Pareto set (Fonseca, 1999) of optimal solutions where no improvement can be made for an objective without sacrificing another objective.

2.2.1. Entities

Features of each entity are described in detail as well as their relationships to other entities.

Trainers. Set T of trainers (teachers) who may be a person or a training institute. As a result, for each event e , an $overload_factor_e \in \mathbb{Z}$ is assigned to indicate the number of simultaneous events

a trainer can attend. For each trainer $t \in T$, a type must be defined (internal, external, etc.) by using $type_t \in TT$, where TT is a set of trainer types. A trainer t can be assigned at minimum to a perimeter (company sector) $PT_t \subseteq P$, where P is a set of perimeters. A trainer t can have a set of individual skills $SK_t \subseteq SK$, where SK is a set of skills.

Rooms. Set R of rooms, with each room $r \in R$ having a minimal and maximal capacity denoted, respectively, $min_cap_r, max_cap_r \in \mathbb{N}$. min_cap_r indicates the minimal number of allocated seats to allocate the room r . max_cap_r denotes the maximum number of allocated seats that r can support. $PR_r \subseteq P$ indicates a room r perimeter (company sector). A room r must have a type (IT room, laboratory room, etc.) denoted by $type_r \in RT$, where RT is a set of room types. A room r can come with a custom configuration (e.g., provides special seats for babies) denoted $config_r \in RC$ where RC is a set of room configurations. Similar to trainers, a room r can have an $overload_factor_r \in \mathbb{Z}$, which denotes the number of simultaneous events it can hold.

Devices. Set EQ of equipment (computer, laser pointer, etc). Each device $eq \in EQ$ has a type $type_{eq} \in EQT$, where EQT is a set of all device types. Like the rest of the resources, a device eq can have an overload factor $overload_factor_{eq} \in \mathbb{Z}$, which provides the maximum number of simultaneous events it can be used at.

Each of the following resources (trainers, devices, or rooms) has a list of unavailability denoted as U_x , where x is a resource.

$$U_x = (d_1, st_1, et_1), \dots, (d_n, st_n, et_n),$$

where $d \in D$ and $st, et \in TS$. Here d denotes a date, D is a set of dates, and TS is a set of time slots with 15 minutes difference. st and et denote the start and end time.

Training. Set E of courses (training or events) where each event $e \in E$ has one of two types (held in an actual room or online) denoted by $type_e \in ET$, where ET is a set of room types. $PE_e \subseteq P$ indicates the perimeter of the course (company sector). A training can have preferred types of resources (for trainers $trainer_type_e \in TT$, for devices $eqt_e \subseteq EQT$ and for rooms $room_type_e \in RT$ or room configuration $room_config_e \in RC$), where TT is a set of trainer types, EQT is a set of equipment types, RT is a set of room types, and RC a set of room configurations.

A $location_e \in L$ indicates where the event e must take place, and L is a set of all locations. A starting and ending hours st_e and $et_e \in TS$ have been defined for the event e as well as a duration in (days or hours) $duration_e \in \mathbb{R}$ are defined for each course e .

Training can indicate preferred dates or days. $SD_e \subseteq D$ (to avoid interruptions by weekends or holidays) or indicate imposed days $ID_e \subseteq D$ (ex: scheduled only on Wednesdays). A minimal and maximal seating capacity is defined per training $min_cap_e \in \mathbb{Z}$, $max_cap_e \in \mathbb{Z}$. If set, min_cap_e indicates the minimal number of registered attendees to consider training e in the scheduling progress. If set, max_cap_e indicates the maximal number of registered attendees the training e can support.

Training can require sometimes advanced trainers, a set of skilled trainers that are associated with event e is provided $ST_e \subseteq T$. In case the associated skilled trainers are not available, trainers with

specific individual skills can be considered. These specific skills of event e are denoted as $SK_e \subseteq SK$, where SK is a set of skills.

Resources can be associated with an event e . We denote associated devices to event e by $AEQ_e \subseteq EQ$, for rooms we use $AR_e \subseteq R$ and trainers $AT_e \subseteq T$.

For each training $e \in E$, sequences can be associated and are indicated by $SEQ_e \subseteq SEQ$. Sequences are considered subevent and are defined in set SEQ . For example, a training (“Learn Soft Skills”) can have two sequences (“Beginner” and “Advanced”), and each must be scheduled in order.

The planner provides a $session_number_e$ that denotes the number of sessions to be planned for this event e and its location $location_e$.

2.2.2. Constraints

Generally, constraints are classified into two types: hard and soft. Hard constraints are rules that must be respected to ensure the feasibility/validity of a timetable.

Soft constraints on the other hand tend to be business/efficiency-oriented, thus responsible for the quality of a schedule. Below we enumerate the hard constraints considered in this work:

- [H1] Events can only be held in rooms having the same type $type_r$ as the event e room type $room_type_e$.
- [H2] Events must take place in rooms with the same location $location_r$ as the event location $location_e$.
- [H3] Only rooms with sufficient capacity can host the training. $min_cap_r \geq min_cap_e$ and $max_cap_r \geq max_cap_e$
- [H4] Devices types must match those indicated by the event.
- [H5] Unavailable resources (trainers, rooms, and devices) cannot be included in the timetable.
- [H6] A trainer cannot be selected for training that requires either individual skills ($SK_t \notin SK_e$) or skilled trainers ($t \notin ST_e$).
- [H7] Each training must be planned within the time frame specified by the planner.
- [H8] Days imposed by an event’s organizational staff must be followed.
- [H9] Events must start on one of their starting days (if specified).
- [H10] No resources (trainer, rooms, and devices) can be placed in overlapping events more than their specified overload factor.
- [H11] Starting/ending times are indicated for each event and must be followed.
- [H12] Sequences are either scheduled in their entirety (in a specific order) or not at all. For each event e has a nonempty set of sequences $SEQ_e \subseteq SEQ$, where $SEQ_e = \{seq_1, seq_2\}$ and $seq_2 \succ seq_1$ if and only if seq_2 is scheduled at after seq_1 .
- [H13] Only resources that share the same perimeter as the event can be selected.
- [H14] The number of scheduled sessions for event e cannot exceed $session_number_e$.

In this work, we consider a timetable that violates no hard constraints as valid. The following soft constraints are considered:

- [S1] A training can indicate a specific room configuration.
- [S2] A training can indicate a specific type of trainer (internal, external, associate, etc.).

- [S3] Resources associated with a training e are preferable over nonassociated ones.
- [S4] Trainings with a duration longer than 1 day must not be interrupted by weekends or long periods of days.
- [S5] A well-balanced event distribution is preferable; to achieve this, each training must adhere to a list of preferred days. The edit distance (Sörensen, 2007) is used to determine how close a solution is to its balanced state. It is expressed as the number of operations required to convert one string to another.

2.2.3. Objectives

Because DiLeap Logistic is owned by several partners, some wish that trainers' workloads be evenly divided, while others want the resource's workload to be maximized. To offer better flexibility, each planner can specify which objectives to be considered before launching the scheduling process.

(O1) Objective 1: Maximize the number of planned events:

$$\max f(x) = \sum_{i=1}^n Nbs_{e_i},$$

where Nbs_e is the total number of successfully scheduled sessions of event e and n is the number of events.

(O2) Objective 2: Minimize the number of soft constraint violations.

$$\min g(x) = \sum_{i=1}^n (NbConf_{n_{e_i}} + NbTrtn_{e_i} + NbARn_{e_i} + NbDistn_{e_i} + NbBdn_e),$$

where n is the number of events, $NbConf_{n_{e_i}}$ is the normalized total number of unique allocated rooms not having the same configuration as required by event $room_type_e$, $Nbtrtn_{e_i}$ shows the normalized total number of unique allocated trainers not having the same type as required by event $trainer_type_e$, $NbARn_{e_i}$ presents the normalized total number of unique allocated resources (trainers, rooms, and equipment) that are not strongly associated with event e , where allocated resources $\notin AT_e, AR_e, AEQ_e$, $NbDistn_{e_i}$ holds the normalized total number of days (distance) between each instance in each session of event e , $NbBdn_e$ is the normalized edit distance between event e schedule and the preferable well-balanced schedule e' .

(O3) Objective 3: Minimize the number of external trainers. This indirectly maximizes internal trainers' workload.

$$\min h(x) = \sum_{i=1}^n NbuT_{e_i},$$

where $NbuT_e$ is the total number of unique external trainers in all scheduled sessions of event e , n is the number of events, and i is the index of a specific event e . This goal cannot be fully realized if the company using the platform relies entirely on external workers. The properties to consider when enabling this objective are explained to planners using the platform.

(O4) Objective 4: Standard deviation will measure the workload deviations between internal trainers. To achieve a fairly balanced workload distribution, we seek to minimize the standard deviation. i denotes the index of event e

$$\min \sigma = \sqrt{\frac{\sum_{i=1}^n (\text{workload}_{t_i} - \overline{\text{mw}})^2}{n-1}},$$

workload_t is the total teaching hours of trainer t having type $\text{type}_t = \text{Internal} \in TT$, n is the total number of unique internal trainers, and $\overline{\text{mw}}$ is the mean workload of all internal trainers and can be described as follows:

$$\overline{\text{mw}} = \frac{\sum_{j=1}^n (\text{workload}_{t_j})}{n}$$

Notice here that **O3** is competing with this objective.

(O5) Objective 5: Minimize the number of classrooms. The goal of this objective is to reduce used classrooms, thus maximizing their utilization and reducing their overall cost (rent and maintenance).

$$\min k(x) = \sum_{i=1}^n NbuR_{e_i},$$

where $NbuR_e$ is the total number of unique rooms in scheduled sessions of event e , n is the number of events, and i denotes the index of event e .

3. Multiobjective evolutionary approach to professional course timetabling

Metaheuristics represent a family of approximate optimization strategies that offer acceptable solutions to complex and difficult problems in a reasonable time. Unlike exact optimization algorithms, metaheuristics do not guarantee that the results are optimal (Hertz and Widmer, 2003).

Among the most well-known metaheuristics are population-based (evolutionary) algorithms that process a large number of candidate solutions at the same time. Among these techniques, GAs are extensively used to solve optimization problems as they are naturally more exploration-oriented due to the large diversity of initial populations. They have been developed by Holland (1973) to mimic the adaptive processes of natural systems. GAs begins with the selection of the problem's chromosome encoding (solution representation). The fittest solutions are chosen for the next iteration after an initial population is generated for evaluation (Sarhani et al., 2023). We will use several popular algorithms found in the literature due to their proven efficiency in dealing with both complex scheduling and non-scheduling problems (Ishibuchi et al., 2016).

3.1. NSGA-II (nondominated sorting genetic algorithm II)

NSGA-II (Deb et al., 2000) is based on the original design of NSGA (Srinivas and Deb, 1994). It consists of ranking and sorting a parent population and generating off-springs with the use of

genetic operators (crossover, mutation, selection). This process is repeated with each generation by using both the rank and crowding distance to select the best solutions.

3.2. *NSGA-III (nondominated sorting genetic algorithm III)*

NSGA-III was designed to handle many objectives (more than two) at the same time (Deb and Jain, 2013). The algorithm is based on reference points (directions) that need to be provided in the initialization phase. NSGA II and III have many similarities regarding ranking and sorting with the use of genetic operators. Although, NSGA III uses a different selection mechanism that relies on associating solutions to reference points before applying a niche preservation operation.

3.3. *MOEA/D (multiobjective evolutionary algorithms by decomposition)*

MOEA/D (Zhang and Li, 2007) is a multiobjective optimization algorithm, based on the idea of decomposing the multiobjective function into several scalar functions through aggregation functions and optimizing each objective in parallel with some effective evolutionary algorithms. The diversity of the population is controlled by the weight vector, which indicates each objective's importance.

3.4. *IBEA (indicator-based evolutionary algorithm)*

IBEA was first suggested by Zitzler and Künzli (2004), where the environmental selection is designed based on a predefined binary indicator. Their work includes results that showcase the performance superiority of IBEA over NSGA-II and SPEA2 using several bi/three objective problems.

3.5. *SPEA-2 (strength Pareto evolutionary algorithm)*

SPEA2 (Zitzler et al., 2001) is an elitist multiobjective evolutionary algorithm that operates with a population of fixed size, from which promising candidates are drawn as parents of the next generation. The resulting offspring then compete with the old ones for inclusion in the population. The various components used in our approach for the selected evolutionary algorithms are listed in the following subsection.

3.6. *Solution representation*

The first step in GAs is encoding or solution representation, which defines a description of a solution's structure. Binary representations are commonly used. For the *MAPT* (Mandarine Academy professional timetabling) problem, we chose a direct representation. Each solution has a set of

Algorithm 1. Constructive heuristic for the MAPT problem: Initial solutions

```

1: Find resources for each training session. in: Einput, session_nbr out: Sol
2: // Assigning resources to sessions.
3: while ( $E_{input} \neq \emptyset$ ) do
4:    $current\_session_i = 0$ ;
5:    $E_{dur\_i} = \text{Event\_duration}(E_i)$ ;
6:    $E_{Seq\_cnt\_i} = \text{Event\_sequence\_count}(E_i)$ ;
7:   while ( $current\_session_i \leq session\_nbr$ ) do
8:     while ( $(E_{dur\_i} \geq 0) \vee (E_{Seq\_cnt\_i} \geq 0)$ ) do
9:        $Event\_day = \text{Valid\_days}(E_i)$ ;
10:       $resources = \text{Valid\_resources}(E_i)$ ;
11:       $session\_instance\_i = (Event\_day, resources)$ ;
12:    end while
13:     $session = session\_instance\_i$ ;
14:     $current\_session_i = current\_session_i + 1$ ;
15:     $E_{dur\_i} = \text{Update\_duration}(E_i, E_{dur\_i})$ ;
16:     $E_{Seq\_cnt\_i} = \text{Update\_sequence}(E_i, E_{Seq\_cnt\_i})$ ;
17:  end while
18:   $Sol = session$ 
19: end while

```

event sessions ES belonging to their respective training. Each session can have one or many session instances si , where selected resources, dates, hours, sequences, and a location are specified.

- Event session instance $si_e = (R_i, d_i, st_i, et_i, l_i, seq_i)$, where
 - R_i denotes the allocated resources set (trainers, rooms, and devices).
 - d_i denotes the allocated day to event session instance si_e .
 - st_i and et_i denote the allocated start and end times to si_e .
 - l_i denotes the allocated location to event session instance si_e .
 - seq_i denotes the specific sequence of the session instance.
- Event session $ES_e = \{si_{1e}, \dots, si_{ne}\}$.
- Event session set $S_e = \{ES_{1e}, \dots, ES_{me}\}$.
- Solution $sol = \{S_{e1}, \dots, S_{ew}\}, \forall e_i \in E$.

3.7. Initial solutions

As mentioned in Sousa et al. (2016), the initial population of GAs can be produced through random, greedy, or heuristic approaches. We opted for a constructive heuristic with hard constraints incorporated into its design. This approach leads to a valid initial population that does not need extra fixing. Our approach is divided into two stages. First, we chose a constructive heuristic that starts with an empty timetable and inserts trainings one at a time. The proposed solutions are then improved by evolutionary algorithms.

The proposed constructive heuristic described in Algorithm 1 uses predefined priority rules assigned to training to determine which one is selected first. The algorithm prioritizes courses with

Algorithm 2. Pseudocode of the $MAPT_{co}$ crossover operator, adapted from Bratković et al. (2009).

```

1: Apply mutation for each solution in:  $P_1, P_2(Parents)$  out:  $O_1, O_2(Off - springs)$ 
2: // initialisation
3:  $N$  = number of events;
4: choose randomly  $E_1$  where  $E_1 > 1$  and  $E_1 \leq N$ ;
5:  $E_2 = N - E_1$ ;
6: // constructing off-springs from parents
7:  $O_1 = \text{Random\_Event}(E_1, P_1) + \text{Random\_Event}(E_2, P_2) \neq P_1$ ;
8:  $O_2 = \text{Random\_Event}(E_2, P_2) + \text{Random\_Event}(E_1, P_1) \neq P_2$ ;

```

longer durations and additional resource constraints. Once the scheduling order is determined, the algorithm begins from the highest priority to the lowest, performing the following steps for each training:

1. Searching for a suitable day, start, and end times that adhere to the following constraints (H7, H8, H9, H11).
2. Searching for a combination of available rooms, trainers, and devices. The following constraints are followed (H1, H2, H3, H4, H5, H6, H10).
3. Repeat for each session while respecting constraint (H14).

If we have a pool of valid choices to choose from, the proposed approach selects random days, time slots, and resources, increasing diversity from one solution to the next. However, if resources/time slots are unavailable, the heuristic discards the affected training session because there are not unlimited resources at each geographical location that meet all defined constraints.

3.8. Crossover

PMX, CX, and OX operators are frequently used to solve scheduling problems (Chinnasri et al., 2012). We propose a new crossover operator called $MAPT_{co}$, which was inspired by Bratković et al. (2009) and two-point crossover. The pseudocode is displayed in Algorithm 2.

$MAPT_{co}$ works as follows: we define N as the number of events found in solution. We then randomly select E_1 and E_2 events from parents one and two and interchange their positions. We define p_c as the crossover probability in our experiments.

Later in our experiments, the proposed crossover operator will be compared to a classical operator (PMX) to determine which is more practical for this specific problem.

A possible conflict with constraints is possible for the aforementioned crossover operators. For example, Parent 1 and Parent 2 do not share the same number of planned sessions per event. During crossover operation, a child solution might end up having certain sessions with unavailable resources since the order was changed. The child solution in this case is not valid. To address this, a repairing function is implemented in which the time slot/resources of the invalid sessions are replaced. If this fails (due to no availability), the infeasible session is discarded.

Table 1
Characteristics of test instances used in our experiments

Size	Small	Medium	Large
<i>Sessions</i>	100	500	1050
<i>Trainings</i>	20	50	70
<i>Duration^a</i>	36.34	97.24	135
<i>Available Days</i>	60	152	244
<i>Trainers</i>	269	269	269
<i>Rooms</i>	230	230	230
<i>Devices</i>	14	14	14
<i>Test Instances</i>	20	20	20

^a Number of days.

3.9. Mutation

A mutation is a chromosomal change that introduces variety into a population and allows it to escape local optima. The mutation, like the crossover, is applied with a predetermined probability p_m . Classical operators (Bit Flip, Swap, and Random Mutation) can be found in the literature.

A custom operator named $MAPT_{mo}$ is proposed to be compared with a classical operator. $MAPT_{mo}$ was adapted from Bratković et al. (2009) and incorporates business logic into its design. The operator randomly selects an event's session and then determines whether to generate a random valid time slot or select new resources (rooms, equipment, or trainers). Two granularity of mutation can be used with $MAPT_{mo}$ that impact the solution's state (small/large changes).

4. Experimental protocol

4.1. Dataset

Mandarine Academy provided us with some historical planning data from Dileap Logistic so that we could test our strategy. A total of 152 different courses were used to generate instances. A summary of our test files can be found in Table 1. Three problem sizes are available: small, medium, and large. As test size increases, more sessions and trainings must be scheduled within a constrained planning window. This was done to see how the algorithms would react if there were more events to satisfy. There are 20 different test instances for each size, and it is important to remember that the duration is expressed in days. As previously mentioned, since this problem differs from academic timetabling, evaluating our approach using academic datasets (like ITC 2007 or ITC 2019) will be inapplicable. This is because our objectives include business requirements and are not just academic, and the main focus of this study is organizational rather than academic.

We published the data we used in our research in Harvard Dataverse, titled “Mandarine Academy Professional Timetabling Dataset” ($MAPTD$) (Hafsa, 2022). $MAPTD$ is a real-world dataset suggested for solving professional timetabling problems, a relatively underexploited scheduling category. However, we believe it can still be used as a useful benchmark dataset to help researchers compare various approaches to traditional timetabling problems (education, health, etc.). When

Table 2
Parameters settings considered for tuning phase

Parameters	Values
<i>POP</i>	50, 75, 100, 125, 150
<i>CX</i>	<i>PMX</i> , <i>MAPT_{co}</i>
<i>CP</i>	0.0, 0.1, 0.2, 0.3, 0.5, 0.8, 1.0
<i>MX</i>	<i>SWAP</i> , <i>MAPT_{mo}</i>
<i>MP</i>	0.1, 0.3, 0.5, 0.7, 0.8, 0.9, 1.0

compared to common educational datasets like ITC2007, MAPT proposes richer features taken from real-world settings to offer insight into the logistics and scheduling challenges of corporate training.

4.2. Parameter tuning

Instead of choosing fixed parameters empirically and applying them to all algorithms indifferently, another protocol is used to provide a fair performance comparison. It uses the i-race package (López-Ibáñez et al., 2016), which implements an iterated racing approach to automatically find optimal settings. Elite configurations are returned based on their average best hypervolume indicator (*HV*) across different test instances. The package focus on improving optimization algorithms and machine learning models. Because the algorithms in our experiments are population-based, they share common parameters:

- Population size (*Pop*): Number of solutions at each generation.
- Crossover operator (*C_o*)
- Crossover probability (*C_p*): Probability that parent solutions will recombine.
- Mutation operator (*M_o*)
- Mutation probability (*M_p*): Probability that a child solution will be mutated.

A list of possible values is chosen for each model parameter, this is shown in Table 2. Parameter values for the population size were chosen based on the number of events in our test instances. The classical PMX crossover operator will be compared to our proposed operator *MAPT_{co}*. The same logic is applied on mutation operators, where the classical Swap mutation is compared to our proposed operator *MAPT_{mo}*.

The final experiments' termination criteria are 4, 8, and 12 hours of computing time. This is the time it takes for algorithms to execute until they are terminated. The duration was chosen to align with the company's interests in running these heavy calculations overnight to provide a solution the next day for complex problems with urgency.

4.3. Performance comparison (*NSGA 2*, *NSGA 3*, *SPEA 2*, *MOEAD/D*, *IBEA*)

For this task, we will be using jMetalPy (Benitez-Hidalgo et al., 2019), a Python framework for single/multiobjective optimization that offers both parallel computing capabilities and easy-to-use

Table 3

I-race results (small instances) using 4 hours as a stopping criterion for three objectives (O1, O2, and O4) and five objectives

OBJ	POP		CX		CP		MX		MP	
	3	5	3	5	3	5	3	5	3	5
<i>NSGAII</i>	125	125	<i>PMX</i>	<i>PMX</i>	0.7	0.7	<i>MAPT_{mo}</i>	<i>MAPT_{mo}</i>	0.2	0.2
<i>NSGAIII</i>	150	75	<i>PMX</i>	<i>PMX</i>	0.8	0.8	<i>MAPT_{mo}</i>	<i>MAPT_{mo}</i>	0.1	0.3
<i>SPEA2</i>	150	150	<i>PMX</i>	<i>PMX</i>	0.8	0.7	<i>MAPT_{mo}</i>	<i>MAPT_{mo}</i>	0.3	0.4
<i>MOEA/D</i>	125	125	<i>PMX</i>	<i>PMX</i>	0.9	0.8	<i>MAPT_{mo}</i>	<i>MAPT_{mo}</i>	0.3	0.4
<i>IBEA</i>	150	125	<i>PMX</i>	<i>PMX</i>	0.6	0.8	<i>MAPT_{mo}</i>	<i>MAPT_{mo}</i>	0.3	0.8

visualization tools. The framework can be used to create custom evolutionary algorithms/operators and define new problems.

5. Results

The initial population used in both the tuning phase as well in experiments was produced by the proposed constructive heuristic. The average number of planned sessions is 100 (100%) for small instances, 498 (98%) for medium instances, and 980 (93.3%) for large instances. These solutions can be found in the folder “Initial Solutions” of the published dataset *MAPTD* (Hafsa, 2022).

5.1. Off-line parameter setting

The hypervolume indicator is used to compute elite configurations provided by i-race for each algorithm (NSGAII, NSGAIII, SPEA2, MOEA/D, and IBEA) with a small problem size. The following nonparametric tests are used on the results: Spearman’s rank correlation coefficient, Kendall’s concordance coefficient, variance, and Friedman’s test (rank).

Table 3 displays the best configuration discovered for each algorithm on small problem instances with three objectives (O1, O2, and O4). The stopping criterion was set to a maximum of 4 hours of computing time. MOEA/D has three additional parameters: neighbor size (*NS*), neighborhood selection probability (*NSP*), and a maximum number of selected solutions (*MSS*). These parameters have the following values: (*NS*: 33, *NSP*: 0.8, and *MSS*: 16). Table 3 displays the best configuration discovered for each algorithm on the small problem instances when all five objectives are used (O1, O2, O3, O4, and O5). The additional parameter values for MOEA/D are *NS*: 15, *NSP*: 0.7, and *MSS*: 8.

Across all returned configurations, *MAPT_{mo}* is chosen, confirming that it outperforms the traditional Swap mutation for this specific problem. After all, *MAPT_{mo}* uses business logic to alter solutions. This was not the case for *MAPT_{co}*, our proposed crossover operator, which was not included in the elite configurations; instead, the classic *PMX* operator was chosen.

Each algorithm will use its best configuration against all problem sizes (small, medium, and large) and with 4, 8, and 12 hours of computing time in the following subsection. This will put the

Table 4

Performance comparison (Mean_{standard deviation}) using small instances and three objectives for over 30 independent runs

Small test instances (three objectives)					
<i>T</i> (hours)	4				
Algorithm	NSGAII	NSGAIII	SPEA2	MOEAD	IBEA
HV	0.89 _{0.026}	0.88 _{0.021}	0.89 _{0.020}	0.73 _{0.021}	0.90 _{0.017}
GD	1.51 _{0.019}	1.52 _{0.011}	1.51 _{0.017}	1.54 _{0.017}	1.55 _{0.019}
IGD	1.38 _{0.041}	1.41 _{0.032}	1.37 _{0.034}	1.50 _{0.036}	1.47 _{0.026}
€	−0.86 _{0.018}	−0.86 _{0.017}	−0.86 _{0.019}	−0.86 _{0.014}	−0.87 _{0.023}
<i>T</i> (hours)	8		12		
Algorithm	NSGAIII	IBEA	NSGAIII	IBEA	
HV	0.94 _{0.012}	0.96 _{0.010}	0.967 _{0.012}	0.963 _{0.009}	
GD	1.53 _{0.020}	1.67 _{0.021}	1.55 _{0.023}	1.68 _{0.014}	
IGD	1.40 _{0.040}	1.58 _{0.042}	1.39 _{0.035}	1.61 _{0.048}	
€	−0.90 _{0.02}	0.95 _{0.010}	−0.93 _{0.024}	−0.95 _{0.011}	

discovered configurations to the test to see how the performance changes when we let algorithms run for longer periods of time against more complex problems.

5.2. Performance analysis on NSGAII, NSGAIII, SPEA2, IBEA, and MOEA/D

We ran 30 independent runs for each algorithm with three objectives (O1, O2, and O4) and five objectives (O1, O2, O3, O4, and O5). We show the results for each problem size (small, medium, and large) with a variety of stopping conditions (4, 8, and 12 hours of computing time).

The main performance metrics used in our experiments are the hypervolume (*HV*) (Zitzler, 1999) to maximize, the generational distance (*GD*) (Van Veldhuizen, 1999), inverse generational distance (*IGD*) (Van Veldhuizen, 1999), and the epsilon-indicator (ϵ) (Zitzler et al., 2003) are to be minimized. For each metric, the average best value and the standard deviation are provided. The Friedman test was used to ensure the statistical relevance of the results.

5.2.1. Results using small instances

Performance analysis using three objectives. Table 4 shows a performance comparison of all algorithms on small problem instances using three objectives.

Starting with 4 hours of computing time and the *HV* metric which is responsible for measuring the convergence and diversity of our solutions. The bold values indicate which algorithm performed better overall for a particular metric and experimental duration.

At first glance, IBEA appears to provide the best results, followed by NSGA II, SPEA2, and NSGA III. The distinction between these algorithms is negligible. When we consider the other performance indicators (*GD*, *IGD*, ϵ), we see no clear advantage from any algorithm, as they all behave similarly, with the exception of MOEA/D, which has the worst results when compared to the others.

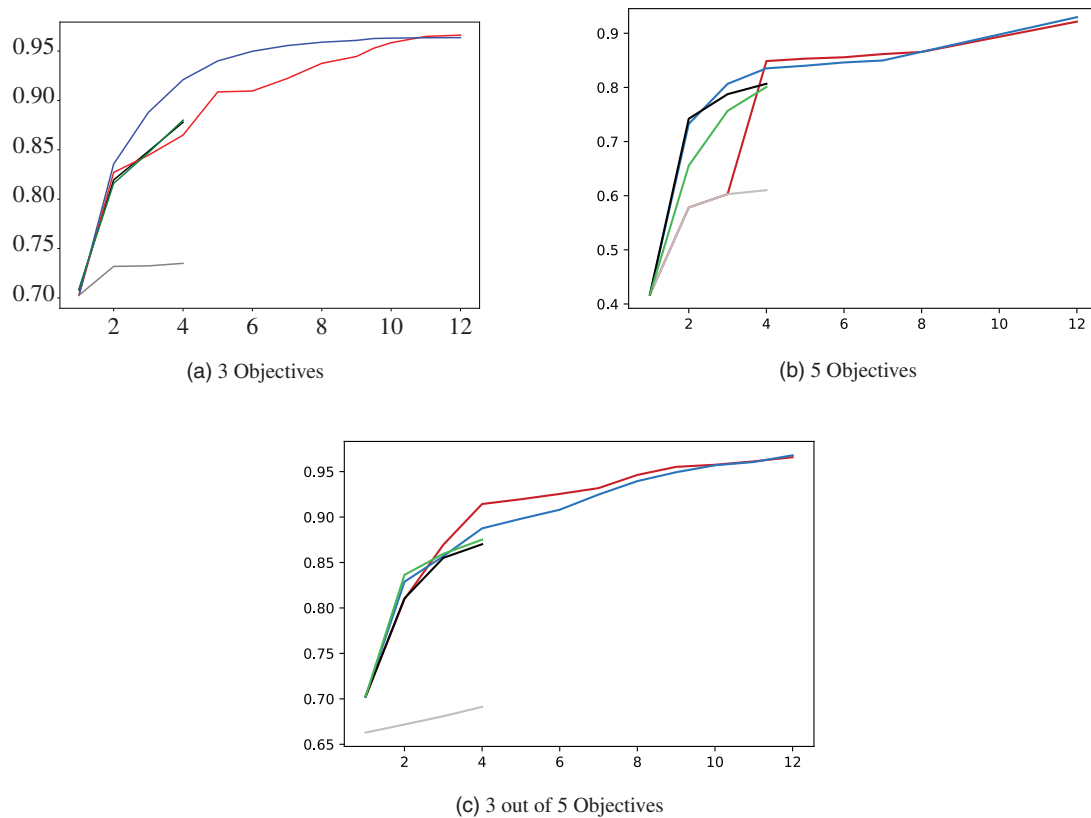


Fig. 1. Evolution of the hypervolume indicator (Y-axis) over 12 hours of computing time (X-axis) using the small problem instances (30 runs) with NSGAIII (red), IBEA (blue), NSGAII (black), SPEA2 (green), and MOEA/D (gray).

After reviewing the previous results, we decided to conduct new experiments with two additional stopping criteria of 8 and 12 hours of computing time. As a result, the algorithms will have more time to improve their results. We chose IBEA and NSGA III for the remaining experiments because they had previously demonstrated good performance.

Moving on, IBEA provides better results in terms of HV with a stopping criterion of 8 hours, while NSGA III trails by a small margin. Taking into account other performance indicators (GD , IGD , ϵ), NSGA III has lower GD and IGD values than IBEA, which had a better ϵ . IBEA maintains its lead in terms of HV for the 12-hour stopping condition, despite a negligible difference. In terms of other performance metrics, NSGA III continues to outperform in terms of GD and IGD , while IBEA maintains its lead with better ϵ and HV .

Convergence analysis using three objectives. Observations from Fig. 1a demonstrate the evolution of the hypervolume indicator for each algorithm. Starting with a 4-hour stopping criterion, most algorithms converged faster, with IBEA and NSGA III leading the way. It is worth noting that the time required to improve the quality of solutions is slower and worse when using MOEA/D, confirming its lower performance when compared to the other algorithms for this specific problem.

Table 5

Performance comparison (Mean_{standard deviation}) using small instances and 5 Objectives for over 30 independent runs

Small test instances (five objectives)					
<i>T</i> (hours)	4				
Algorithm	NSGAII	NSGAIII	SPEA2	MOEAD	IBEA
HV	0.80 _{0.026}	0.83 _{0.021}	0.80 _{0.020}	0.61 _{0.021}	0.85 _{0.017}
HV (3 OBJ)	0.87 _{0.016}	0.88 _{0.021}	0.87 _{0.015}	0.68 _{0.037}	0.91 _{0.014}
GD	1.90 _{0.014}	1.95 _{0.016}	1.91 _{0.015}	1.99 _{0.034}	2.00 _{0.015}
IGD	1.71 _{0.037}	1.76 _{0.042}	1.71 _{0.040}	1.96 _{0.039}	1.90 _{0.007}
ϵ	−0.83 _{0.023}	−0.85 _{0.021}	−0.83 _{0.021}	−0.76 _{0.029}	−0.88 _{0.019}
<i>T</i> (hours)	8		12		
Algorithm	NSGAIII	IBEA	NSGAIII	IBEA	
HV	0.89 _{0.029}	0.91 _{0.014}	0.92 _{0.018}	0.91 _{0.016}	
HV (3 OBJ)	0.93 _{0.026}	0.95 _{0.010}	0.96 _{0.016}	0.95 _{0.012}	
GD	2.01 _{0.039}	2.15 _{0.030}	2.05 _{0.034}	2.16 _{0.020}	
IGD	1.79 _{0.062}	2.04 _{0.077}	1.84 _{0.068}	2.07 _{0.083}	
ϵ	−0.91 _{0.029}	−0.95 _{0.009}	−0.94 _{0.014}	−0.95 _{0.010}	

Despite starting with a similar performance, IBEA was able to converge faster than NSGA III. From 8 to 12 hours of computing time, we can see that NSGA III is still converging while IBEA is slowly stabilizing (stagnate).

Performance analysis using five objectives. Table 5 compares the performance of all algorithms on small problem instances using five objectives. Starting with 4 hours of computing time, IBEA produces the best results, followed by NSGA III in the second place. SPEA2 and NSGA II have comparable results (third place). We aggregated the same three objectives from the previous experiment (O1, O2, and O4) to see if aggregating produces better results than running experiments with only three objectives. The results found in *HV(3OBJ)* differ slightly from those found in Table 4.

Other performance indicators (*GD*, *IGD*, ϵ) show no clear advantage from any algorithm, with the exception of MOEA/D, which has the worst results when compared to the others. NSGA II maintained the lowest *GD* across both experiments, while SPEA 2 maintained a lower *IGD* even after switching to five objectives. IBEA is the clear winner by a small margin for both three and five objectives experiments in the 4-hour time frame.

For 8 hours, NSGA III is behind IBEA by a small difference in terms of *HV*. The same results are present in *HV* for three objectives aggregated from the total five objectives. For the 12-hours experiments, better results are provided by NSGA III even when the difference is not huge. This is also true for *HV(3OBJ)*. Taking into account other performance indicators (*GD*, *IGD*, ϵ) for 8 and 12 hours, NSGA III has better *GD* and *IGD* compared to IBEA which was able to only obtain a better ϵ .

IBEA has a slight lead over NSGA III in terms of *HV* after 8 hours. The same results can be found in *HV* for three of the five objectives combined. NSGA III produces better results for the 12-hour experiments, even if the difference is not significant. This also applies to *HV(3OBJ)*. Taking

Table 6

Performance comparison (Mean_{standard deviation}) using medium instances and three objectives for over 30 independent runs

Medium test instances (three objectives)						
<i>T</i> (hours)	4		8		12	
Algorithm	NSGAIII	IBEA	NSGAIII	IBEA	NSGAIII	IBEA
HV	0.68 _{0.003}	0.67 _{0.006}	0.68 _{0.004}	0.70 _{0.010}	0.70 _{0.005}	0.73 _{0.013}
GD	1.50 _{0.0008}	1.50 _{0.001}	1.50 _{0.001}	1.49 _{0.001}	1.49 _{0.007}	1.48 _{0.004}
IGD	1.42 _{0.034}	1.38 _{0.021}	1.39 _{0.021}	1.34 _{0.016}	1.37 _{0.025}	1.33 _{0.012}
ε	−0.72 _{0.012}	−0.73 _{0.006}	−0.73 _{0.005}	−0.74 _{0.007}	−0.74 _{0.006}	−0.74 _{0.008}

into account other performance indicators (*GD*, *IGD*, ϵ) for 8 and 12 hours, NSGA III has lower *GD* and *IGD* than IBEA, which achieved the best ϵ score.

Convergence analysis using five objectives. By looking at the convergence graphs in Fig. 1b for five objectives and Fig. 1c for three out of five objectives using small problem instances, we can see that at 4 hours, each algorithm is still converging, with IBEA and NSGA III leading. In the middle of the experiments, the gap between IBEA and NSGA III widens (between 4–8 hours). After about 12 hours of computing time, we see IBEA's performance stabilizing while NSGA III continues to improve and eventually surpasses IBEA at the end of the experiments. It appears that 8 hours of computing time already yields a good *HV*, so the extra 4 hours may not be necessary after all.

Now that we have seen both algorithms tested under different computing times, we will put them under further testing with medium test instances to see if their performance can withstand more complex problems.

5.2.2. Results using medium instances

Performance analysis using three objectives. Table 6 compares the performance of NSGA III and IBEA on medium problem instances with three objectives over 4, 8, and 12 hours.

Starting with 4 hours, NSGA III was able to maintain a higher *HV* despite the small difference with IBEA. Even after adding more computing time (8 hours), neither algorithm's performance changed significantly except that IBEA has slightly outperformed NSGA III this time. The final (12-hour) experiments show a very small performance increase that barely justifies the extra 4 hours of computing time. IBEA dominates medium instances experiments because it has the best values for every metric.

When compared to Table 4, which uses small problem instances, both algorithms struggle to achieve higher performance when faced with more complex problem instances. This can be caused by a variety of factors, including the fact that both algorithms may need to go through a tuning phase using medium instances to find suitable configurations.

Convergence analysis using three objectives. Convergence graphs of IBEA and NSGA III are shown in Fig. 2a for three objectives and durations of 4, 8, and 12 hours. Both algorithms are still

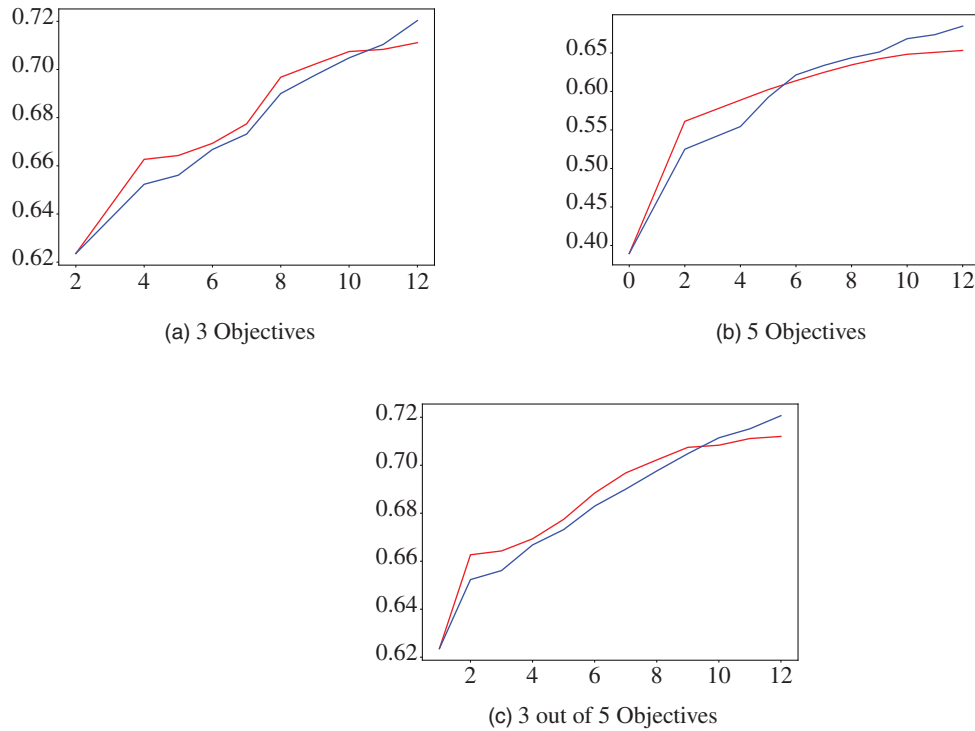


Fig. 2. Evolution of the hypervolume indicator (Y-axis) over 12 hours of computing time (X-axis) using the medium problem instances (30 runs) with NSGAIII (red) and IBEA (blue).

Table 7

Performance comparison (Mean_{standard deviation}) using medium instances and five objectives for over 30 independent runs

Medium test instances (five objectives)						
T (hours)	4		8		12	
Algorithm	NSGAIII	IBEA	NSGAIII	IBEA	NSGAIII	IBEA
HV	0.59 _{0.015}	0.55 _{0.015}	0.63 _{0.015}	0.65 _{0.015}	0.65 _{0.016}	0.68 _{0.010}
HV (3 OBJ)	0.66 _{0.005}	0.64 _{0.002}	0.70 _{0.010}	0.69 _{0.012}	0.71 _{0.018}	0.72 _{0.011}
GD	1.88 _{0.004}	1.90 _{0.002}	1.87 _{0.004}	1.90 _{0.001}	1.87 _{0.004}	1.91 _{0.002}
IGD	1.80 _{0.020}	1.83 _{0.005}	1.79 _{0.014}	1.85 _{0.008}	1.79 _{0.017}	1.87 _{0.006}
ε	-0.70 _{0.007}	-0.66 _{0.010}	-0.69 _{0.009}	-0.70 _{0.011}	-0.69 _{0.016}	-0.70 _{0.013}

converging even after more time is added in between. While NSGA III maintained the lead for the first 10 hours, IBEA outperforms the latter and achieves a better performance in 12 hours.

Performance analysis using five objectives. Table 7 compares the performance of NSGA III and IBEA on medium problem instances with five objectives over 4, 8, and 12 hours.

Table 8

Performance comparison (Mean_{standard deviation}) using large instances and three objectives for over 30 independent runs

Large test instances (three objectives)				
T (hours)	4		8	
Algorithm	NSGAIII	IBEA	NSGAIII	IBEA
HV	0.60 _{3.15e-05}	0.60 _{2.75e-05}	0.63 _{0.03}	0.64 _{0.009}
GD	1.48 _{0.0001}	1.48 _{3.26e-07}	1.47 _{0.008}	1.47 _{0.0009}
IGD	1.47 _{6.66e-16}	1.47 _{6.66e-16}	1.42 _{0.047}	1.37 _{0.008}
ε	-0.68 _{3.61e-05}	-0.68 _{3.15e-05}	-0.70 _{0.021}	-0.71 _{0.009}
T (hours)	12		24	
Algorithm	NSGAIII	IBEA	NSGAIII	IBEA
HV	0.66 _{0.007}	0.66 _{0.009}	0.68 _{0.008}	0.70 _{0.009}
GD	1.46 _{0.002}	1.47 _{0.001}	1.44 _{0.01}	1.45 _{0.002}
IGD	1.38 _{0.016}	1.36 _{0.010}	1.34 _{0.01}	1.33 _{0.01}
ε	-0.72 _{0.006}	-0.72 _{0.006}	-0.72 _{0.004}	-0.73 _{0.007}

NSGA III maintains superiority over IBEA in the 4-hour range and has achieved the best results in every metric, similar to the results obtained using only three objectives in Table 6. Both algorithms achieve a very small increase in *HV* when computing time reaches 8 hours.

While IBEA was able to defeat NSGA III when comparing the *HV* of all five objectives, NSGA III had a slightly better *HV* when aggregating three of the five objectives. In terms of the remaining metrics, NSGA III has the best scores in terms of *GD*, *IGD*, and ϵ . We can see a slow increase in *HV* for both algorithms in the final experiments with 12 hours of computing time. NSGA III has better *GD* and *IGD*, while IBEA has slightly lower ϵ and *HV*. Even if performance is not at the same level as when we used both algorithms on the small test instances, both algorithms are still improving.

Convergence analysis using five objectives. The hyper-volume evolution graphs shown in Fig. 2b and Fig. 2c for five and three out of five objectives, respectively, demonstrate how IBEA was able to keep improving over time, eventually surpassing NSGA III. We also notice that both algorithms exhibit similar behavior in the final half of the experiments, where they dramatically slow their rate of improvement. This also implies that both algorithms require additional computing time because they are still converging, and thus better performance may be possible.

5.2.3. Results using large instances

Performance analysis using three objectives. The performance of IBEA and NSGA III using large problem instances and three objectives for 4, 8, 12, and 24 hours is shown in Table 8.

Similar performance behavior was observed with medium instances in Table 6. Because both algorithms use their best configurations based on tests applied only to small instances, this could confirm that the performance of both algorithms is problem-size dependent. NSGA III and IBEA

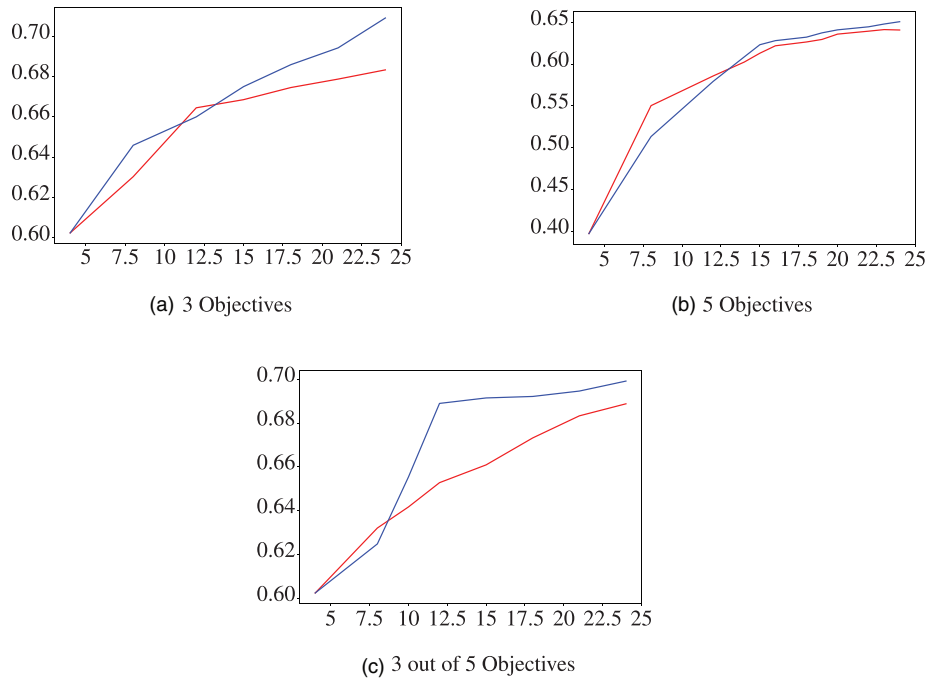


Fig. 3. Evolution of the hypervolume indicator (Y-axis) over 24 hours of computing time (X-axis) using the large problem instances (30 runs) with NSGAIII (red) and IBEA (blue).

have similar HV scores on two separate occasions (4 and 12 hours). For the remaining metrics, both algorithms had the same values or, in rare cases, an advantage over the other.

In terms of HV and ϵ , IBEA is the clear winner in the 24-hour experiments, while NSGA III has better GD and IGD . Despite the small difference, both algorithms are improving. As previously stated, both algorithms may require a tuning phase using large instances to find elite configurations that perform well.

Convergence analysis using three objectives. Because both algorithms did not improve their initial HV for the entire computing time allowed (4 hours), the results found in Table 8 for three objectives are the initial and final scores.

The HV evolution of IBEA and NSGA III is depicted in Fig. 3a. Because NSGA III and IBEA are still converging, they both require additional computing time. While NSGA III was attempting to close the gap with IBEA, it was successful after about 12 hours of computing time, but IBEA quickly regains the lead and surpasses NSGA III.

Performance analysis using five objectives. Table 9 compares the performance of NSGA III and IBEA on large problem instances with five objectives over 4, 8, 12, and 24 hours.

NSGA III achieved the best HV results (for 4 and 8 hours) and comparable results with IBEA on 12 hours of computing time. While NSGA III dominated HV when three objectives were ag-

Table 9

Performance comparison (Mean_{standard deviation}) using large instances and five objectives for over 30 independent runs

Large test instances (five objectives)				
<i>T</i> (hours)	4		8	
Algorithm	NSGAIII	IBEA	NSGAIII	IBEA
HV	0.46 _{0.07}	0.39 _{1.89e-05}	0.57 _{0.02}	0.49 _{0.08}
HV (3 OBJ)	0.61 _{0.015}	0.60 _{2.91e-05}	0.63 _{0.008}	0.61 _{0.011}
GD	1.89 _{0.006}	1.90 _{7.33e-07}	1.88 _{0.005}	1.89 _{0.001}
IGD	1.86 _{0.036}	1.89 _{1.33e-15}	1.81 _{0.019}	1.86 _{0.027}
ε	-0.68 _{0.018}	-0.66 _{2.22e-16}	-0.70 _{0.008}	-0.68 _{0.013}
<i>T</i> (hours)	12		24	
Algorithm	NSGAIII	IBEA	NSGAIII	IBEA
HV	0.59 _{0.01}	0.59 _{0.02}	0.64 _{0.009}	0.65 _{0.010}
HV (3 OBJ)	0.65 _{0.008}	0.69 _{0.010}	0.68 _{0.009}	0.69 _{0.009}
GD	1.87 _{0.004}	1.89 _{0.002}	1.88 _{0.005}	1.90 _{0.003}
IGD	1.80 _{0.022}	1.84 _{0.008}	1.81 _{0.019}	1.87 _{0.006}
ε	-0.70 _{0.008}	-0.69 _{0.010}	-0.690 _{0.008}	-0.693 _{0.010}

gregated, IBEA achieved a higher *HV* when more computing time was available. NSGA III took first place in each experiment for the remaining performance metrics (*GD*, *IGD*, and ϵ). IBEA outperforms NSGA in terms of *HV* for five objectives, *HV* for three objectives, and ϵ for 24-hour experiments. Unfortunately, using medium and large instances revealed that our chosen elite configurations will not perform optimally unless given more computing time, which is impractical.

Convergence analysis using five objectives. Figures of hypervolume evolution found in Fig. 3b and c demonstrate the performance of both algorithms. In five objective experiments, IBEA struggles to improve against NSGA III and it improves at a slower rate than NSGA III. Up until about 10 hours, NSGA III outperformed IBEA in terms of convergence rate, after which IBEA took the lead. When only three objectives (out of five) were considered, NSGA III led for more than half of the experiment time and maintained close readings when IBEA overtook it.

When faced with small test problems, both experiments with three or five objectives demonstrated exceptional performance; however, this was not the case for medium and large instances, in which IBEA and NSGA III were always able to provide a variety of solutions. It is always possible to make improvements, and repeating tuning experiments with consideration for more complex issues may produce outcomes that differ from what has already been observed.

6. Conclusions

In this paper, we propose a solution for the professional course timetabling problem encountered at Mandarin Academy by using popular multiobjective evolutionary algorithms (NSGA II, NSGA III, SPEA 2, MOEA/D, and IBEA). A mathematical formulation of our objective functions is

presented along with entities and hard/soft constraints involved in the scheduling procedure. The real-world problem was partly captured as some elements were simplified to extract only essential features of the planning problem. This was done to reduce the complexity of the MAPT problem while still operating within the confines of a timetabling problem. However, this simplification affects only information about resources such as company hierarchy levels, skill levels, and course details that are not fully described as they are relative to each company requirement. All the other elements (constraints and objectives) were captured completely and reflect real-world planning conditions for Mandarin Academy. MAPTD (Hafsa, 2022) intended to assess timetabling solvers in education and other professional areas is now available for researchers.

To generate valid timetables, a constructive heuristic has been developed. The heuristic provided a high-quality initial population that adheres to specified hard constraints and was able to plan the majority of the trainings in each test instance. Unfortunately, solutions lack diversity which was expected since the heuristic generates only feasible solutions, and providing diversified solutions is not always possible.

The construction heuristic was able to schedule more sessions with no broken hard constraints, while the greedy technique only manages to plan a fraction of sessions, with multiple broken hard constraints. While both run times are significantly less than the manual technique, the construction heuristic falls short in this case since it must validate the correctness of offered solutions, while the greedy approach does not, making the greedy approach faster. The provided results are approved by platform managers as they respect defined constraints and provide an operationally feasible solution minimizing time spent fixing broken solutions, unlike the greedy approach.

Our suggested mutation operator, $MAPT_{mo}$, outperformed the traditional *Swap* operator in a tuning phase using all mentioned algorithms on the small problem instances. For the crossover operator, PMX was chosen based on its performance compared to our proposed operator $MAPT_{co}$.

Experimental results show that both IBEA and NSGA III gave the best results in terms of hypervolume, generational distance, inverse generational distance, and ϵ -indicator. By using evolutionary algorithms, we were able to obtain more diversified solutions that are significantly better than their initial state.

According to experimental tests, IBEA and NSGA III both produced the best results in terms of ϵ -indicator, generational distance GD , inverse generational distance IGD , and hypervolume HV . We were able to produce more diversified solutions that were significantly superior to their initial state by using evolutionary algorithms.

When observing convergence plots, a promising improvement can be made by considering the additional computational time in our experiments in order for the algorithms to handle complex instances. Considering the research aspect alone, it is possible to add more computing power and additional time. This is impractical from a business standpoint because it could result in delays or higher computational costs. The use of MOEA/D in our experiments turned out to be a bad investment since it only gave the worst results. This confirms that this algorithm was not suited for this specific problem. The proposed method has replaced the previous version of automatic planning. Our method presents solutions graphically, allowing the planner to confirm or reject proposed sessions. A dashboard that displays information about planned sessions mobilized resources, and pending actions also make it easier for clients to stay up to date on the overall training process.

Among our approach's proposals is the addition of filters for each client who uses the platform. These filters are useful both before and after optimization (multicriteria decision-making). Clients

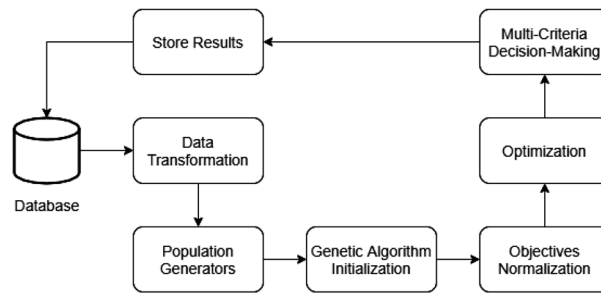


Fig. 4. Proposed approach architecture.

would initially have the option of selecting preferred objectives and whether they want to maximize or minimize them. Furthermore, the ability to choose which constraints (hard and soft) to activate would provide greater flexibility in a variety of scenarios.

Finally, after providing Pareto optimal solutions, planners are given an interface with modifiable weights. These weights represent the degree of preference for each objective. The previous propositions aim to provide better usability of our approach and explore its full potential. This offers more freedom for planners to further tune their requirements. The overall architecture of the proposed approach is shown in Fig. 4.

For future work, we are considering a different experimental protocol that includes parameter tuning using medium and large instances rather than just small test problems. By using parameters adapted to problem size, we might see a different winner in terms of algorithm performance. Testing random initialization as an alternative to the construction heuristic would be another strategy to see how it might impact the planning process.

References

- Bellio, R., Ceschia, S., Di Gaspero, L., Schaerf, A., Urli, T., 2016. Feature-based tuning of simulated annealing applied to the curriculum-based course timetabling problem. *Computers & Operations Research* 65, 83–92.
- Benitez-Hidalgo, A., Nebro, A.J., Garcia-Nieto, J., Oregi, I., Del Ser, J., 2019. jmetalpy: a Python framework for multi-objective optimization with metaheuristics. *Swarm and Evolutionary Computation* 51, 100598.
- Bratković, Z., Herman, T., Omrcen, V., Čupić, M., Jakobović, D., 2009. University course timetabling with genetic algorithm: a laboratory exercises case study. In *Evolutionary Computation in Combinatorial Optimization*. Springer, Berlin, pp. 240–251.
- Burke, E.K., De Causmaecker, P., Petrovic, S., Berghe, G.V., 2002. A multi-criteria meta-heuristic approach to nurse rostering. In *Proceedings of the 2002 Congress on Evolutionary Computation*, Vol. 2. IEEE, Piscataway, NJ, pp. 1197–1202.
- Burke, E.K., Kendall, G., Soubeiga, E., 2003. A tabu-search hyperheuristic for timetabling and rostering. *Journal of Heuristics* 9, 6, 451–470.
- Burke, E.K., McCollum, B., Meisels, A., Petrovic, S., Qu, R., 2007. A graph-based hyper-heuristic for educational timetabling problems. *European Journal of Operational Research* 176, 1, 177–192.
- Caprara, A., Fischetti, M., Toth, P., 2002. Modeling and solving the train timetabling problem. *Operations Research* 50, 5, 851–861.

- Chinnasri, W., Krootjohn, S., Sureerattanan, N., 2012. Performance comparison of genetic algorithm's crossover operators on university course timetabling problem. In 2012 8th International Conference on Computing Technology and Information Management (NCM and ICNIT), Vol. 2. IEEE, Piscataway, NJ, pp. 781–786.
- Christofides, N., 1976. The vehicle routing problem. *Revue française d'automatique, informatique, recherche opérationnelle*, 10, 1, 55–70.
- Coello, C.A.C., Cortés, N.C., 2005. Solving multiobjective optimization problems using an artificial immune system. *Genetic Programming and Evolvable Machines* 6, 2, 163–190.
- Coello, C.A.C., Dhaenens, C., Jourdan, L., 2010. Multi-objective combinatorial optimization: problematic and context. In *Advances in Multi-Objective Nature Inspired Computing*. Springer, Berlin, pp. 1–21.
- Czibula, O., Gu, H., Russell, A., Zinder, Y., 2017. A multi-stage IP-based heuristic for class timetabling and trainer rostering. *Annals of Operations Research* 252, 2, 305–333.
- Deb, K., Agrawal, S., Pratap, A., Meyarivan, T., 2000. A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: NSGA-II. In *PPSN*. Springer, Berlin, pp. 849–858.
- Deb, K., Jain, H., 2013. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints. *IEEE Transactions on Evolutionary Computation* 18, 4, 577–601.
- Derigs, U., Jenal, O., 2005. A GA-based decision support system for professional course scheduling at Ford Service Organisation. *OR Spectrum* 27, 1, 147–162.
- Eley, M., 2006. Ant algorithms for the exam timetabling problem. In Burke, E.K., Rudová, H. (eds), *Practice and Theory of Automated Timetabling VI (PATAT 2006)*. Lecture Notes in Computer Science, Vol. 3867. Springer, Berlin, pp. 364–382.
- Fonseca, C., Fleming, P., 1999. Genetic Algorithms for Multiobjective Optimization: Formulation Discussion and Generalization the fifth Intl Conference on Genetic Algorithms, 93.
- Haase, K., Latteier, J., Schirmer, A., 1998. The course scheduling problem at Lufthansa Technical Training. *European Journal of Operational Research* 110, 3, 441–456.
- Hafsa, M., 2022. Mandarin Academy Professional Timetabling Dataset, Harvard Dataverse. <https://doi.org/10.7910/DVN/A4JU5E> (accessed on September 23, 2022).
- Hafsa, M., Wattedled, P., Jacques, J., Jourdan, L., 2021. A multi-objective evolutionary approach to professional course timetabling: A real-world case study. In *IEEE Congress on Evolutionary Computation (CEC)*, Piscataway, NJ, pp. 997–1004. doi: 10.1109/CEC45853.2021.9504797
- Hertz, A., Widmer, M., 2003. Guidelines for the use of meta-heuristics in combinatorial optimization. *European Journal of Operational Research* 151, 2, 247–252.
- Holland, J.H., 1973. Genetic algorithms and the optimal allocation of trials. *SIAM Journal on Computing* 2, 2, 88–105.
- Hoos, H., Jourdan, L., Kessaci, M.E., Stützle, T., Veerapen, N., 2022. Preface to the special cluster on stochastic local search: recent developments and trends. *International Transactions in Operational Research* 29, 5, 2735–2736.
- Ishibuchi, H., Imada, R., Setoguchi, Y., Nojima, Y., 2016. Performance comparison of NSGA-II and NSGA-III on various many-objective test problems. In *IEEE Congress on Evolutionary Computation (CEC)*, Piscataway, NJ, pp. 3045–3052. doi: 10.1109/CEC.2016.7744174.
- Lewis, R., 2008. A survey of metaheuristic-based techniques for university timetabling problems. *OR Spectrum* 30, 1, 167–190.
- López-Ibáñez, M., Dubois-Lacoste, J., Cáceres, L.P., Birattari, M., Stützle, T., 2016. The i-race package: iterated racing for automatic algorithm configuration. *Operations Research Perspectives* 3, 43–58.
- dos Santos, P.T.G., Borenstein, D., 2024. Multi-objective optimization of the maritime cargo routing and scheduling problem. *International Transactions in Operational Research* 31, 1, 221–245.
- Sarhani, M., Voß, S., Jovanovic, R., 2023. Initialization of metaheuristics: comprehensive review, critical analysis, and research directions. *International Transactions in Operational Research* 30, 6, 3361–3397.
- Sörensen, K., 2007. Distance measures based on the edit distance for permutation-type representations. *Journal of Heuristics* 13, 1, 35–47.
- Sousa, T., Morais, H., Castro, R., Vale, Z., 2016. Evaluation of different initial solution algorithms to be used in the heuristics optimization to solve the energy resource scheduling in smart grids. *Applied Soft Computing* 48, 491–506.

- Srinivas, N., Deb, K., 1994. Multiobjective optimization using nondominated sorting in genetic algorithms. *Evolutionary Computation* 2, 3, 221–248.
- Van Veldhuizen, D.A., 1999. Multiobjective evolutionary algorithms: classifications, analyses, and new innovations. Technical report, AFIT.
- Zhang, Q., Li, H., 2007. MOEA/D: a multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11, 6, 712–731.
- Zhu, J., Li, X., 2011. An effective meta-heuristic for no-wait job shops to minimize makespan. *IEEE Transactions on Automation Science and Engineering* 9, 1, 189–198.
- Zitzler, E., 1999. *Evolutionary Algorithms for Multiobjective Optimization: Methods and Applications*, Kluwer Academic Publishers, Dordrecht, Vol. 63.
- Zitzler, E., Künzli, S., 2004. Indicator-based selection in multiobjective search. In PPSN, Springer, Berlin, pp. 832–842.
- Zitzler, E., Laumanns, M., Thiele, L., 2001. Spea2: improving the strength Pareto evolutionary algorithm. *TIK-report* 103. Swiss Federal Institute of Technology Zurich.
- Zitzler, E., Thiele, L., Laumanns, M., Fonseca, C.M., Da Fonseca, V.G., 2003. Performance assessment of multiobjective optimizers: An analysis and review. *c* 7, 2, 117–132.